

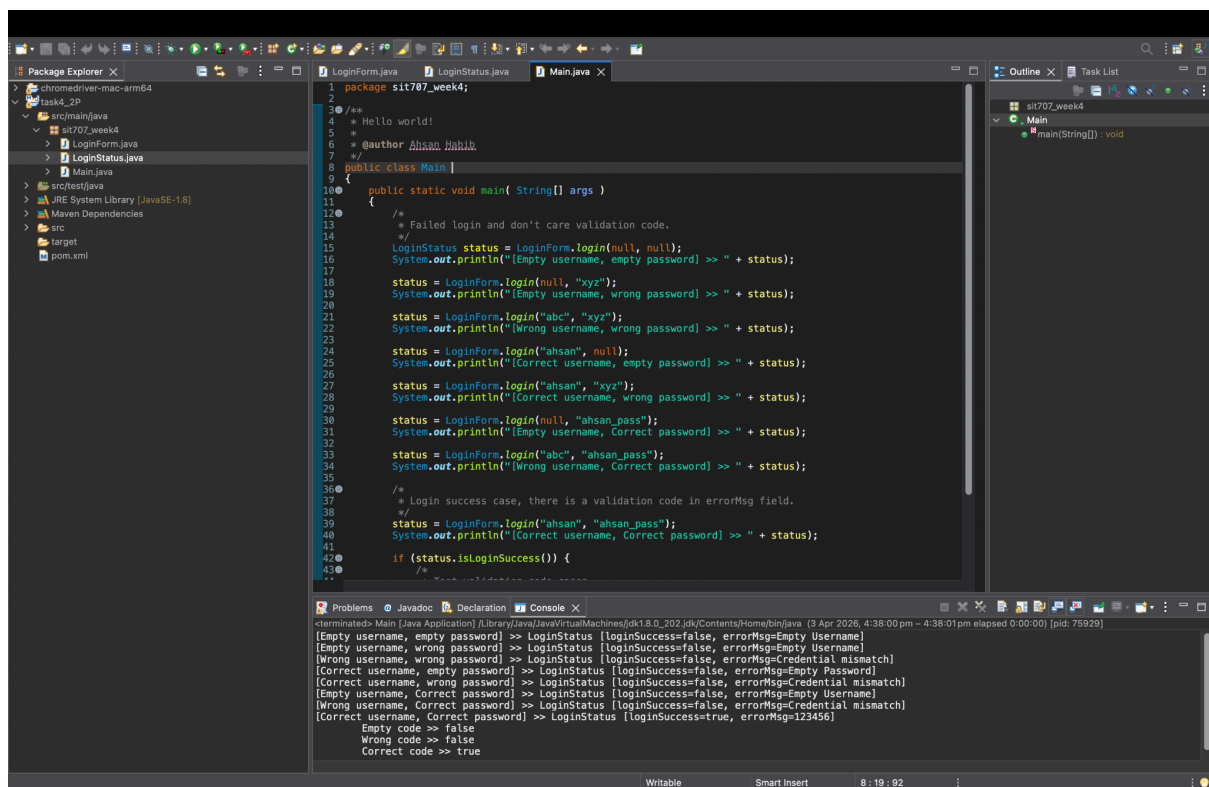
Task 4.1P — Software Testing Submission

Student Name: Ayush Indapure

Student ID: 224880003

This task was about understanding and testing a login system using decision tables and JUnit test cases. It involved analysing different combinations of inputs like username, password, and validation code. Based on that, we created multiple test cases to verify if the system behaves correctly in all scenarios.

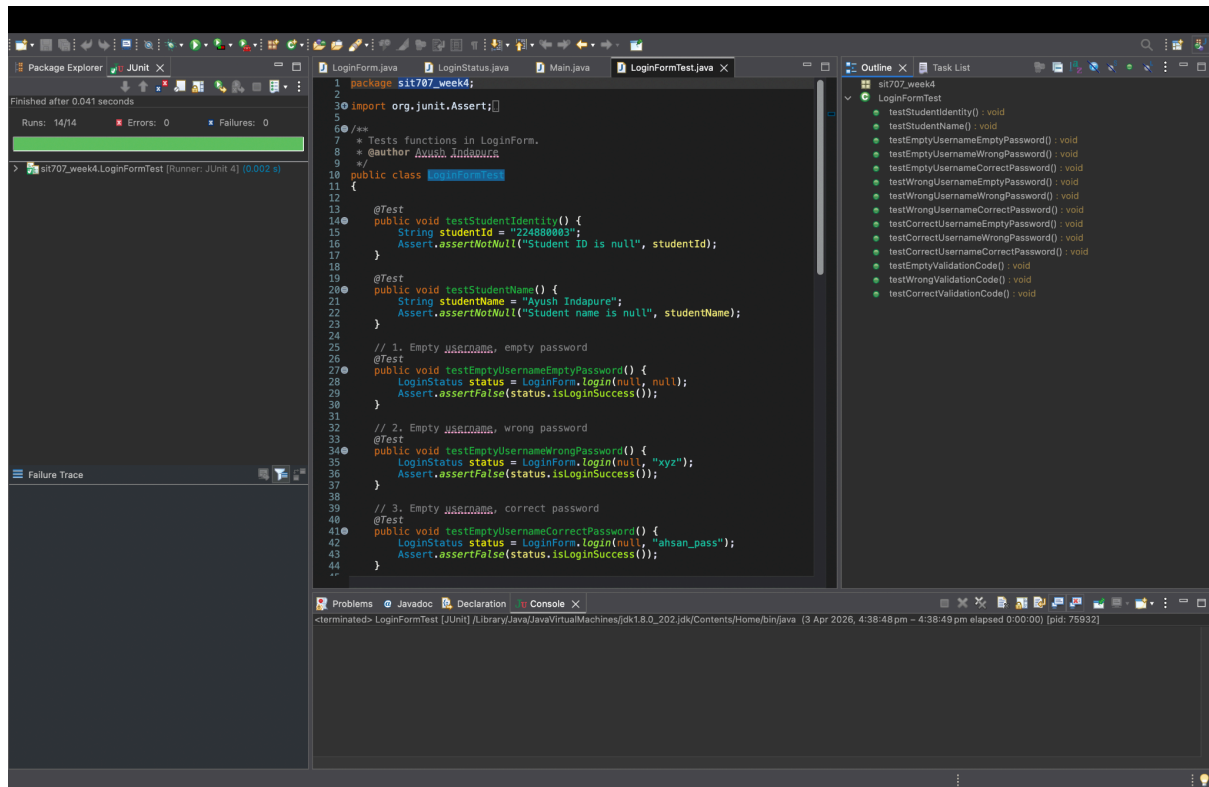
Screenshot of JUnit tab which shows test statistics including Runs, Errors and Failures



The screenshot displays an IDE with the following components:

- Package Explorer:** Shows the project structure with packages like `src/main/java` and `src/test/java`. Files include `LoginForm.java`, `LoginStatus.java`, and `Main.java`.
- Editor:** Displays the `Main.java` file. It contains a `main` method that tests the `LoginForm.login` method with various inputs (null, empty, wrong, correct) and checks the `loginStatus` and `errorMsg` fields. Comments indicate the expected outcomes for each test case.
- Outline:** Shows the `Main` class with the `main` method.
- Console:** Displays the output of the program, showing the results of the login attempts and the status of the login process. The output includes messages like `[Empty username, empty password] >> LoginStatus [loginSuccess=false, errorMsg=Empty Username]` and `[Correct username, Correct password] >> LoginStatus [loginSuccess=true, errorMsg=123456]`.

Screenshot of eclipse console which shows the output



My programs source code for tests (LoginFormTest.java)

```
package sit707_week4;
```

```
import org.junit.Assert;
```

```
import org.junit.Test;
```

```
/**
```

```
 * Tests functions in LoginForm.
```

```
 * @author Ayush Indapure
```

```
 */
```

```
public class LoginFormTest
```

```
{
```

```
    @Test
```

```
    public void testStudentIdentity() {
```

```
        String studentId = "224880003";
```

```
        Assert.assertNotNull("Student ID is null", studentId);
```

```
    }
```

```
    @Test
```

```
    public void testStudentName() {
```

```
        String studentName = "Ayush Indapure";
```

```
        Assert.assertNotNull("Student name is null", studentName);
```

```
}
```

```
// 1. Empty username, empty password
```

```
@Test
```

```
public void testEmptyUsernameEmptyPassword() {
```

```
    LoginStatus status = LoginForm.login(null, null);
```

```
    Assert.assertFalse(status.isLoginSuccess());
```

```
}
```

```
// 2. Empty username, wrong password
```

```
@Test
```

```
public void testEmptyUsernameWrongPassword() {
```

```
    LoginStatus status = LoginForm.login(null, "xyz");
```

```
    Assert.assertFalse(status.isLoginSuccess());
```

```
}
```

```
// 3. Empty username, correct password
```

```
@Test
```

```
public void testEmptyUsernameCorrectPassword() {
```

```
    LoginStatus status = LoginForm.login(null, "ahsan_pass");
```

```
    Assert.assertFalse(status.isLoginSuccess());
```

```
}
```

// 4. Wrong username, empty password

@Test

```
public void testWrongUsernameEmptyPassword() {  
  
    LoginStatus status = LoginForm.login("abc", null);  
  
    Assert.assertFalse(status.isLoginSuccess());  
  
}
```

// 5. Wrong username, wrong password

@Test

```
public void testWrongUsernameWrongPassword() {  
  
    LoginStatus status = LoginForm.login("abc", "xyz");  
  
    Assert.assertFalse(status.isLoginSuccess());  
  
}
```

// 6. Wrong username, correct password

@Test

```
public void testWrongUsernameCorrectPassword() {  
  
    LoginStatus status = LoginForm.login("abc", "ahsan_pass");  
  
    Assert.assertFalse(status.isLoginSuccess());  
  
}
```

// 7. Correct username, empty password

@Test

```
public void testCorrectUsernameEmptyPassword() {  
    LoginStatus status = LoginForm.login("ahsan", null);  
    Assert.assertFalse(status.isLoginSuccess());  
}
```

// 8. Correct username, wrong password

@Test

```
public void testCorrectUsernameWrongPassword() {  
    LoginStatus status = LoginForm.login("ahsan", "xyz");  
    Assert.assertFalse(status.isLoginSuccess());  
}
```

// 9. Correct username, correct password

@Test

```
public void testCorrectUsernameCorrectPassword() {  
    LoginStatus status = LoginForm.login("ahsan", "ahsan_pass");  
    Assert.assertTrue(status.isLoginSuccess());  
}
```

// ---- VALIDATION CODE TESTS ----

// 10. Empty validation code

@Test

```
public void testEmptyValidationCode() {  
    boolean result = LoginForm.validateCode(null);  
    Assert.assertFalse(result);  
}
```

// 11. Wrong validation code

@Test

```
public void testWrongValidationCode() {  
    boolean result = LoginForm.validateCode("abcd");  
    Assert.assertFalse(result);  
}
```

// 12. Correct validation code

@Test

```
public void testCorrectValidationCode() {  
    boolean result = LoginForm.validateCode("123456");  
    Assert.assertTrue(result);  
}
```

```
}
```

Fair bit of discussion about Decision table testing and unit test creation using JUnit.

Decision Table Testing

Decision table testing is a black-box software testing technique used to test website/siftwares/tools behaviour for different combinations of inputs. It is particularly useful when the output depends on multiple conditions acting together.

In this task, the **LoginForm** class has three key variables that determine the outcome:

Initial decision table from the results I got

Conditions / Actions	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11
Username	-	-	-	W	W	W	C	C	C	C	C
Password	-	W	C	-	W	C	-	W	C	C	C
Validation Code	X	X	X	X	X	X	X	X	-	W	C
-----	----	----	----	----	----	----	----	----	----	----	----
Login Success	F	F	F	F	F	F	F	F	T	T	T
Validation Result	X	X	X	X	X	X	X	X	F	F	T

- **Username** — can be empty (–), wrong (W), or correct (C)
- **Password** — can be empty (–), wrong (W), or correct ©

- **X - can be Dont care**
- **T - can be True**
- **F - can be True**
- **Validation Code** — can be empty (–), wrong (W), or correct (C) — only relevant after a successful login

By creating a decision table with all meaningful combinations of these three variables, we can systematically identify every test case needed for full coverage. Columns 1 to 8 all result in login failure, meaning the validation code is irrelevant (don't care — X). Only columns 9 –11 represent a successful login where the validation code is then tested.

This approach ensures no important combination is missed and makes the expected outcome explicit before any test code is written, reducing the chance of incomplete testing.

Unit Test Case Creation Using JUnit

JUnit is a widely used Java testing framework that allows developers to write and run automated unit tests. Each test is an independent method annotated with `@Test`, which JUnit automatically discovers and executes.

In this task, each column of the decision table maps directly to one `@Test` method in `LoginFormTest.java`. For example:

- **Column 1** (empty username, empty password) → `testEmptyUsernmeEmptyPassword()` calls `LoginForm.login(null, null)` and asserts the result is `false`
- **Column 9** (correct username, correct password, empty code) → `testCorrectUsernameCorrectPassword()` asserts login returns `true`, then `testEmptyValidationCode()` asserts `validateCode(null)` returns `false`

The `Assert` class provides methods to verify outcomes:

- `Assert.assertTrue()` — confirms a condition is true

- `Assert.assertFalse()` — confirms a condition is false
- `Assert.assertNotNull()` — confirms a value is not null (used for student identity tests)

Running all 14 tests with **Runs: 14/14, Errors: 0, Failures: 0** confirms that the `LoginForm` class behaves correctly across all decision table combinations.